

Agentic AI落地方法论：MCP化·Skill·Harness·Platform

内部方法论文档 | 聂小尘

版本：v0.3 Working Draft

核心命题

AI的生成能力是公共品。GPT-4、Claude、豆包、Kimi——所有人都能调同一个模型，获得同一级别的原始生成能力。

用好AI的能力才是壁垒。

具体来说，有两类知识当前被锁死在个人大脑中，无法沉淀、无法复用、无法scale：

- Skill知识**——"怎么让AI做好某件具体的事"。一个资深教研员知道怎么让AI生成一份好的学情报告，但这套判断逻辑只存在于他的脑子里和他的聊天记录里。换一个人做，质量断崖式下降。
- Harness知识**——"怎么通过多轮迭代把AI的产出从60分打磨到90分"。一个好的运营知道第三轮该收紧约束、第五轮该换角度、第七轮该引入对比——但这套节奏感和判断力没有任何记录，下次做类似的事要从头摸索。

我们的方法论就是解决这个问题。但在讲方法论之前，需要先建立一个前提认知。

前提：范式转换——从"给人用的系统"到"给Agent用的系统"

为什么大多数AI落地项目是"简单加法"

今天市场上绝大多数"AI+行业"方案的做法是：拿一个现有系统，在上面加一层对话框或者接一个大模型API。

ERP系统 + ChatGPT = "智能ERP"。教务系统 + 大模型 = "AI教学平台"。

这是**简单加法**。本质上，底层系统的逻辑没有任何改变——它依然是为人设计的：界面是给人看的，流程是给人点的，数据是按人的操作习惯组织的。AI只是被强行嫁接在上面，充当一个"更聪明的搜索框"。

这种加法的天花板非常低。因为AI能做的事情，被限制在了"人原来怎么用这个系统"的框架内。

MCP化：让系统的能力面向Agent

真正的Agentic AI落地需要一个范式转换：**系统的能力不仅要面向人，还要面向Agent。**

MCP (Model Context Protocol) 的意义不是一个技术协议，而是一种设计思维的转变：

传统设计思维：我有一个系统，人通过UI来操作它。AI可以帮人更快地填表单、更好地写内容、更智能地搜索——但操作主体还是人。

MCP化的设计思维：我有一套业务能力，这些能力被拆解成原子级的服务单元。人可以通过UI调用它们，Agent也可以通过协议调用它们。Agent不是"代替人去点界面"，而是直接操作底层能力。

能力原子化：深耕领域的架构逻辑

MCP化不只是"给系统加API"。它要求对业务能力做一次根本性的重新梳理——**能力原子化**。

举一个教育场景的例子：

传统教务系统的能力是按"页面"组织的：成绩录入页、学情报告页、家长通知页。每个页面是一整块功能，面向人的操作流程设计。

能力原子化之后，这些变成了独立的、可被Agent调用的能力单元：

- 查询学生课堂表现(学生ID, 时间范围) → 返回结构化数据
- 生成学情分析(课堂数据, 评价维度, 输出风格) → 返回报告
- 发送家长通知(学生ID, 内容, 渠道) → 执行动作
- 获取课标对齐建议(知识点, 学段) → 返回建议

每一个原子能力都是独立的、可组合的、可被Skill编排的。Agent可以自由地把它们串起来完成复杂任务，而不是被迫模拟人在页面上的点击流程。

为什么这是前提而不是可选项

如果不做MCP化，Skill层就只能编排"人的操作步骤"——本质上是RPA，是自动化脚本，是脆弱的。界面改一下就全断了。

做了MCP化之后，Skill层编排的是"业务能力"——跟界面无关、跟具体系统的实现无关。这才是真正可持续的Agentic AI落地架构。

简单来说：**MCP化决定了你的Agentic AI方案是"加了一层皮"还是"重构了骨架"。**

深耕一个领域的正确姿势

当我们说"深耕一个垂直领域"的时候，积累的不应该是"一套方案"，而应该是一个**领域的能力原子库**。

十个教育客户的需求各不相同——有的要精准教学，有的要家校沟通，有的要教研支撑。但底层的能力原子（学情分析、课标对齐、行为观测、通知分发）是高度共享的。

方案是项目制的，每个都不同；能力原子是产品级的，可以跨客户复用。

这也是为什么MCP化不只是一个技术决策，而是一个商业模式决策：**你是在做项目，还是在建能力资产？**

方法论四层总览

在MCP化这个前提之上，方法论分为三个能力层：

层级	定位	解决的问题	一句话
MCP化	前提	能力怎么暴露	把面向人的系统能力拆解为面向Agent的原子服务
Skill	能力层	做什么	把隐性的领域know-how编码为可声明、可复用、可组合的显性单元
Harness	质量层	怎么做好	把多轮打磨的方向感和质量判断标准外化为可审计的工程结构
Platform	运行层	让谁都能做	提供运行环境，让能力原子、Skill和Harness在组织中流转

第一层：Skill——固化"做什么"

概念

Skill不是prompt模板。Prompt模板是"填空题"， Skill是"决策逻辑"。

一个prompt模板：

“请根据以下课堂记录， 为{学生姓名}生成一份学情分析报告， 包含{维度1}、{维度2}……”

一个Skill：

编码了"什么样的课堂行为信号值得关注"、"怎么判断学生的参与度等级"、"何时应该给鼓励性评语vs建设性建议"、"输出格式应该适配家长阅读还是教师教研"等一系列领域判断逻辑。同时， 它知道在什么时候调用哪些能力原子（查询课堂数据、生成分析、发送通知）， 以及如何组合它们。

区别在于：prompt模板告诉AI"输出什么格式"， Skill告诉AI"怎么思考这个问题并调度正确的能力来解决它"。

Skill与MCP化的关系： MCP化提供了可调用的能力原子， Skill定义了如何编排这些原子。没有MCP化， Skill只能编排"文本生成"；有了MCP化， Skill可以编排"查询→分析→判断→行动"的完整业务链路。

Skill的关键特征

- 可声明 (Declarative)：** 不依赖人去口头解释"怎么用"， Skill自身包含完整的触发条件、输入要求、行为逻辑和质量标准。
- 可组合 (Composable)：** 复杂任务通过多个Skill组合完成。学情分析Skill + 家长沟通话术Skill + 数据可视化Skill = 一次完整的家校沟通流程。
- 可迭代 (Evolvable)：** Skill本身也是需要打磨的。用了十次之后发现某个判断逻辑不对， 改Skill本身， 所有后续调用同步升级。知识积累是组织级的， 不是个人级的。
- 可审计 (Auditable)：** 可以review一个Skill的逻辑是否合理， 而不是review一个人的prompt是否写得好。前者是工程问题， 后者是玄学问题。

Skill的本质

Skill是把行业专家的判断力编码为可执行的AI指令。

这意味着：Skill的质量上限不取决于AI能力，而取决于领域专家是否参与了Skill的设计。AI是执行引擎，Skill是驾驶手册。

第二层：Harness——固化"怎么做好"

概念

Harness解决的是一个被严重低估的问题：单次调用AI不够好，怎么办？

现实中，绝大多数有价值的AI应用都不是一轮问答能完成的。你需要多轮迭代：生成初版 → 审查 → 补充约束 → 再生成 → 换角度 → 精修 → 终版。

问题在于：这个迭代过程中的每一步判断——"这一版哪里不够好"、"下一轮应该加什么约束"、"什么时候该停下来"——都是无形的认知劳动。它不被记录、不被复盘、不被传承。

Harness就是把这个过程显性化的工程框架。

Harness的四个维度

1. 观测性 (Observability)

每一轮AI的输入、输出、以及人的判断依据，都应当是可观测的。不是事后翻聊天记录，而是结构化地记录"这一轮为什么要这么改"。

这解决的是"事后复盘"问题：一个月后回头看，你还能理解当时的迭代思路。

2. 委派架构 (Delegation Architecture)

哪些决策AI可以自主做，哪些必须人来拍板，边界在哪里？

这不是一个技术问题，是一个治理问题。类似于企业管理中的授权体系——你不会让一个新来的主管直接签百万合同，AI也一样。Harness定义了这个授权边界。

3. 反馈回路 (Feedback Loop)

AI的产出质量应该随着使用次数提升，而不是每次从零开始。这需要结构化的反馈机制：什么样的产出被接受了、什么样的被拒绝了、拒绝的原因是什么。

这些反馈信号应该回流到Skill的迭代中，形成一个持续优化的闭环。

4. 质量判断标准的外化

"好不好"不能只存在于某个人的感觉中。Harness要求把质量判断标准显性化：这份报告达到了什么水平、差在哪里、为什么差。

这让质量控制从"找一个经验丰富的人把关"变成"任何人都能执行的checklist"。

Harness的本质

Harness是把人在多轮迭代中的隐性判断力编码为可复用的流程结构。

如果说Skill固化的是"领域专家的大脑"， Harness固化的是"优秀操作者的手感"。

第三层：Platform——固化"让谁都能做"

概念

有了Skill和Harness， 还需要一个运行环境让它们脱离特定个人， 在组织中跑起来。

Platform解决的核心问题：从"某个人会用AI"到"整个组织能用AI"。

Platform的关键能力

- **Skill的管理与分发**：哪些Skill可以用、谁能用、谁能改、版本怎么管理。类似代码仓库对于软件团队的意义。
- **Harness的运行与监控**：多轮迭代的流程能被系统承载， 而不只是"某个人在ChatGPT窗口里手动来回跑"。每一轮的输入输出可追溯。
- **引擎可插拔**：底层模型不绑定。今天用Claude， 明天需求变了换GLM或MiniMax， Skill和Harness不用重写。这在国内合规环境和成本优化场景下尤其关键。
- **权限与合规**：谁能看到什么数据、AI的输出是否需要人工审批、操作记录是否留痕。对于有监管要求的行业（教育、金融、医疗）， 这不是加分项， 是入场券。

Platform的本质

Platform是让Skill和Harness成为**组织能力**而不是**个人能力**的基础设施。

交付模式：Skill构建服务（FDE模式）

为什么客户自己建不了Skill

方法论讲完了四层架构， 一个自然的问题是：**谁来做？**

客户自己做不了Skill构建。原因很简单——这件事需要两种能力的交叉：

- **领域判断力**：知道"什么样的课堂行为值得关注"、"报价单里哪些条款是合规红线"、"哪种库存模式意味着该补货了"。这些知识在客户的领域专家脑子里。
- **AI能力建模**：知道如何把上述判断逻辑转化为Skill的声明结构、触发条件、质量标准， 以及如何与MCP化后的能力原子对接。这些能力在我们这里。

领域专家不懂怎么把自己的判断逻辑"编程化"——他甚至可能没意识到自己在做判断。纯AI团队不懂业务——他们能写prompt， 但写不出好的Skill， 因为他们不知道什么算"好"。

****Skill构建必须在两者的交叉地带完成。 ****这就是我们提供的FDE（Field Deployment Engineering）式服务的价值所在。

FDE模式的工作方式

我们的FDE不是传统意义上的"驻场实施", 而是**知识提取与能力编码的专业服务**。工作流程分为四个阶段：

阶段一：观察与浸泡

FDE嵌入客户的实际业务场景，跟领域专家（老张们）一起工作。不是采访，是并肩做事。在这个过程中观察：他们怎么用AI、哪些判断是靠经验、哪些步骤是隐性的、哪些迭代是习惯性的但从未被记录。

核心产出：一份**隐性知识地图**——列出了所有"在人脑子里但从未被写下来"的判断逻辑。

阶段二：抽象与建模

把观察到的隐性知识抽象为Skill结构。这是整个过程中技术含量最高的环节——需要判断：哪些逻辑应该是一个独立Skill、哪些应该是Skill内的判断分支、Skill之间的组合关系是什么、触发条件如何定义、质量标准如何量化。

同时，评估现有系统的MCP化需求：哪些能力原子需要从现有系统中暴露出来，才能让Skill编排真正的业务流程而不只是文本生成。

核心产出：**Skill架构蓝图 + MCP化改造清单**。

阶段三：构建与打磨

实际构建Skill，在真实业务场景中反复迭代。这个阶段的关键是：让领域专家参与Skill的质量评判。FDE写Skill、专家判断产出好不好、FDE根据反馈调整Skill逻辑——这个循环本身就是在实践Harness方法论。

核心产出：**可运行的Skill库 + 配套的Harness流程**。

阶段四：移交与赋能

把Skill库和Harness流程部署到Platform上，培训客户团队使用和维护。关键目标：客户应该有能力自己迭代Skill（改判断逻辑、加新场景），而不是每次都需要我们介入。

核心产出：**组织化运行的Agentic AI能力系统 + 客户团队的Skill维护能力**。

FDE模式的商业逻辑

这个模式的商业价值在于：

对客户而言——他买到的不只是一个平台或一套工具，而是"把他最优秀员工的判断力系统化"的专业服务。这是他自己做不到的，也是市面上大多数AI公司不提供的。

对我们而言——每一次FDE服务都是在积累领域能力。服务教育客户A时构建的Skill，80%的判断逻辑在教育客户B那里也适用。服务的客户越多，能力原子库和Skill库越厚，后续服务的边际成本越低、交付速度越快。

这也是为什么FDE模式不是成本项，而是资产积累过程。每一个FDE项目都在给能力资产库添砖加瓦。

FDE与四层方法论的关系

方法论层级	FDE在其中的角色
MCP化	评估现有系统，设计能力原子暴露方案
Skill	核心交付——从领域专家脑中提取知识，编码为Skill
Harness	在Skill打磨过程中顺带建立迭代流程规范
Platform	将产出部署到运行环境，完成组织化移交

双轨价值叙事

同一套方法论，面向两种不同的客户，每一层的意义完全不同。

A轨：面向产业Solution Provider

他是做方案卖给别人的人：系统集成商、行业SaaS公司、AI应用开发商、数字化转型服务商。他的核心焦虑：同质化——你用GPT包一层，我也用GPT包一层，凭什么你赢？

他的处境

他面对的市场现实是：底层模型人人可用，API调用门槛极低。如果你的方案只是"包了一层UI + 接了一个大模型"，那任何一个竞争对手都能在一个月内做出功能类似的产品。

客户也越来越精明——"你这个东西跟我直接用ChatGPT有什么区别？我为什么要付你的钱？"

他需要回答的问题只有一个：我的方案里有什么是别人抄不走的？

三层方法论在A轨的意义

MCP化 → 你做的不是"加法"，是"重构"

市场上大多数竞争对手在做的事情是：拿一个现成系统，上面糊一层AI。客户问"这跟我直接用ChatGPT有什么区别"，他们答不上来。

你做的事情不同：你把客户的业务能力拆成了原子服务，让AI能直接操作业务本身，而不是模拟人去点界面。这从架构层面就跟"糊一层AI"拉开了代差。

更重要的是：你在一个垂直领域深耕时，积累的是一个不断增长的能力原子库。服务第一个客户时打磨出的能力原子，服务第十个客户时还能复用。竞争对手要追赶，追的不是你的代码，是你的行业能力积累。

Skill → 你的护城河

你的方案里不只是"接了一个API"，而是编码了行业know-how。竞争对手拿到同一个模型、抄了你的UI、甚至挖走你的工程师——但他拿不走你的Skill库。因为Skill里封装的是你跟客户共同打磨出来的行业判断逻辑，这是靠时间和场景积累的，不是靠技术抄的。

护城河不在模型层，在Skill层。

Harness → 你的质量背书

你能跟客户说"我的方案不是撞大运出好结果"。每一次AI的输出都经过了结构化的迭代流程，有观测、有校验、有回路。这是工程化的质量保证，不是"我们的prompt engineer很厉害"。

客户续约的理由不是"你的demo做得漂亮"，而是"用了半年，质量一直稳定"。Harness是质量稳定性的来源。

Platform → 你的利润率

你服务第一个客户时打磨出的Skill和Harness，在服务第二个同行业客户时可以80%复用。服务十个客户不需要十倍人力。

更重要的是：平台的可插拔架构意味着你不会被某个模型供应商锁死。今天客户要求用国内模型，你切一下引擎就行，不用重写方案。这是投标时的硬实力。

FDE服务 → 你的Skill库怎么来

你可能认同"Skill是护城河"，但下一个问题是：谁来建这些Skill？你的工程师懂技术但不懂行业，客户的专家懂行业但不懂AI能力建模。

这就是我们提供FDE式Skill构建服务的意义——我们的人嵌入到你的项目中，跟你的行业专家一起工作，把隐性知识提取出来编码成Skill。你获得的不只是"一个能跑的方案"，而是一个持续增值的Skill资产库。

对于A轨客户来说，还有一层更深的价值：**我们帮你建立自己的Skill构建能力**。第一期我们的FDE主导，你的团队跟着学；第二期你的团队主导，我们的FDE辅助；第三期你自己就能做了。这不是锁客，是赋能——但你会继续用我们的Platform，因为Skill跑在上面。

A轨的关键话术

"你的竞争对手也能调GPT的API。你凭什么赢？

第一，他们做的是加法——在现有系统上糊一层AI。你做的是重构——把业务能力原子化，让AI能真正操作业务。

第二，你在这个行业里打磨了两年的判断逻辑，现在全在团队某几个人的脑子里。我们帮你把这些逻辑工程化、产品化，变成你方案的一部分。

人可以走，能力不会走。新客户来了，你不用从零开始。"

A轨的决策者画像

- 关心"如何在投标中胜出"
- 关心"如何提高项目毛利"
- 关心"如何降低对核心人员的依赖"
- 关心"如何跟客户讲一个不一样的故事"

B轨：面向自己做产业的企业

他是自己干活的人：工厂老板、学校校长、贸易公司负责人、连锁门店管理者。他的核心焦虑：人的瓶颈——能干的人太贵、教人太慢、人一走能力就断。

他的处境

他可能已经在用ChatGPT / 豆包了，甚至用得还不错。但他发现一个问题：

- 他自己用得好，但没法让员工也用得好
- 他教了员工怎么用，员工换个场景就不会了
- 每次用都是从头开始，没有积累
- 他不知道员工用得到底好不好，没法管控质量
- 业务量翻倍了，用AI的人手也得翻倍，scale不起来

他的本质需求不是"用上AI"——他已经用上了。他的需求是**"让AI的能力从我个人的经验变成公司的系统能力"**。"

四层方法论在B轨的意义

MCP化 → 你现有的系统不用扔掉，但需要"开个口子"

很多产业企业一听"AI落地"就以为要换系统。不是的。MCP化做的事情是给你现有的系统"开接口"——你的ERP、你的进销存、你的教务系统，该用的继续用。我们做的是让AI能读懂这些系统的数据、调用这些系统的能力，而不是替代它们。

对企业来说，这意味着改造成本极低：不是"推倒重来"，而是"在现有基础上开放能力"。

Skill → 老张的经验不再只是老张的

老张是公司里用AI用得最好的人。他知道怎么让豆包写出合规的报价单，他知道怎么让ChatGPT分析客户需求。但老张的这些经验只存在于他的大脑和他的聊天记录里。

Skill做的事情：跟老张一起，把他的判断逻辑提取出来，编码成可复用的单元。然后小王、小李都能用同一套Skill，做出接近老张水平的产出。

老张退休了，能力还在。新人入职，不用从头教。

Harness → 不教员工"怎么跟AI聊天"，给他们一条路走

"你去问问AI"是最没用的赋能方式。员工不知道问什么、不知道怎么判断好坏、不知道不好的时候怎么改。

Harness给员工的不是"AI使用培训"，而是一条结构化的路径：先做什么、再做什么、到这一步检查什么、不达标怎么调整。把80分的操作流程固定下来，员工跟着走就行。

结果：上手时间从"培训一个月"变成"看流程半天"。

Platform → 管理层看得见、控得住

谁在用、用了多少、产出质量怎么样、有没有在变好——管理层应该能看到这些数据，而不是把AI使用变成一个"员工自己折腾"的黑箱。

同时，Platform确保合规：AI不会碰到不该碰的数据、输出不会跳过该有的审核、操作全部留痕。

对于有监管要求的行业，这意味着能用AI，同时不踩红线。

FDE服务 → 我们来帮你"把老张的脑子掏出来"

你可能在想：道理我都懂，但谁来做这件事？让老张自己把经验写下来？他写不出来——不是不愿意，是隐性知识就是这样，做的时候自然而然，让他总结反而说不清楚。让IT部门做？他们不懂业务判断。

这就是我们的FDE团队做的事。我们的人会跟老张一起工作一段时间——不是采访他，是跟他一起干活。在这个过程中，观察他的判断逻辑、提取他的经验、编码成系统能用的Skill。老张不需要写任何文档，他只需要像平时一样工作，然后告诉我们"这个行，那个不行"就够了。

B轨的关键话术

"你现在用豆包/ChatGPT已经能做不少事了。
但你有没有发现——你会用，不代表你的团队会用？
你摸出来的经验，锁在你自己的脑子里，没法变成公司的能力？
我们做的事情，就是帮你把'你会用AI'变成'你的公司会用AI'。
你的经验变成系统能力，新人来了直接上手，你不用一个个教。"

B轨的决策者画像

- 关心"怎么让员工也能用好AI"
- 关心"核心人员离职后能力不断档"
- 关心"业务翻倍但不想翻倍招人"
- 关心"数据安全、合规、可控"

双轨对照表

维度	A轨 · Solution Provider	B轨 · 产业企业
核心焦虑	同质化，方案没壁垒	人的瓶颈，能力不能scale
MCP化的意义	架构代差：从"糊一层"到"重构能力"，积累能力原子库	低成本接入：不换系统，给现有系统"开接口"
Skill的意义	护城河：抄不走的行业know-how	资产化：个人经验变组织能力
Harness的意义	质量背书：可审计的工程化交付	降门槛：员工不用自己摸索

维度	A轨 · Solution Provider	B轨 · 产业企业
Platform的意义	利润率：跨客户复用、引擎不锁定	可管控：透明、合规、可度量
FDE服务的意义	帮你建Skill库，同时培养你自己的Skill构建能力	帮你把"老张的脑子"变成公司资产
关键词	差异化、壁垒、毛利	不依赖人、可复制、可管控
核心一句话	"你的方案里有什么是别人抄不走的？"	"你个人的AI能力怎么变成公司的？"
打动点	赢单、提利、建壁垒	省人、提效、降风险
典型客户	系统集成商、行业SaaS、数字化服务商	工厂、学校、贸易公司、连锁门店

附：方法论的底层逻辑

为什么是MCP化 + Skill + Harness + Platform这四层？

因为Agentic AI落地需要解决四个递进的问题：

问题	当前状态	对应层级	目标状态
能力如何被调用	锁在面向人的UI里	MCP化	拆解为Agent可调用的原子服务
领域判断力	在专家脑子里	Skill	编码为可执行的AI指令
迭代打磨力	在操作者手感里	Harness	外化为可复用的流程结构
组织运营力	在管理者经验里	Platform	承载为可度量的系统能力

四层之间有严格的依赖关系：

- 没有MCP化， Skill只能编排文本生成， 无法驱动业务流程
- 没有Skill， Harness只是空壳流程， 不知道"做什么"
- 没有Harness， Platform跑起来的东西质量不可控
- 没有Platform， 前三层的能力锁在个人和项目里， 无法组织化

这四层的共同指向是：把隐性的、个人的、不可传承的能力， 变成显性的、组织的、可积累的资产。
这就是为什么我们不只是"做AI应用"， 也不只是"做AI平台"——我们做的是Agentic AI能力的组织化工程。